

Laya as a Real-Time Probabilistic Decision Engine for Football: A Preregistered Multi-Agent Evaluation Protocol

[AUTHOR NAME]*

[AFFILIATION]

September 26, 2026

Study protocol v2.0.0 — Registered-report stage 1 (design paper; no results).
Preregistration: [OSF DOI] · Code: [GITHUB URL] · Preprint: [ARXIV ID]

Abstract

AI agents built on large language models are increasingly deployed as decision engines that return full probability distributions on demand, yet their quality as *live* probabilistic forecasters is rarely measured with the rigor applied to statistical models. We present a preregistered evaluation protocol (v2.0.0) for Laya, a typed, multilingual AI decision engine, as a real-time probabilistic forecaster of football match outcomes. At every snapshot—taken at seven fixed cutoffs (pre-match and minutes 15, 30, 45, 60, 75, 85) and immediately after salient events (goals, red cards, penalties, substitutions)—Laya emits distributions over the final 1X2 result, 17 score buckets, total corners (0–20, 21+), and total yellow cards (0–12, 13+). The protocol enforces point-in-time integrity: automated leakage tests on 100 % of snapshots, forbidden-field and provenance checks, and three information tiers (A: pre-match; B: minimal live; C: full live). Laya is compared with baselines receiving *identical* information—uniform, historical frequency, Poisson, Dixon–Coles, multinomial logistic regression, and bookmaker odds—under a strict temporal split, with a stratified bootstrap clustered by match (at least 2,000 replicates) and Holm–Bonferroni correction, against five preregistered hypotheses (H1–H5). This article reports the protocol only; data collection, execution, and results follow in stage 2. The reference implementation is open source (MIT); the text is CC-BY 4.0.

Keywords: probabilistic forecasting; calibration; large language models; sports analytics; preregistration; evaluation protocol; multi-agent systems.

1 Introduction

Good decisions in dynamic environments require continuously updated *distributions*, not one-off point estimates. A decision-maker monitoring a live process needs $P(\text{outcome} \mid \text{information available now})$, refreshed every time new information arrives, and needs to know whether those probabilities can be trusted—that is, whether they are calibrated, sharp, and reactive to events. The statistical literature provides mature tools for judging such forecasts: proper scoring rules [1, 7], calibration–sharpness diagnostics [8, 17], and clustered inference for dependent observations [2].

A new class of systems now claims a place in this pipeline: AI decision engines built on large language models (LLMs) that return structured probability distributions over typed outcome spaces, on demand and in natural language. Their measured quality as *repeated, in-play probability assessors* is, however, largely

*Corresponding author: [AUTHOR NAME], [AFFILIATION]. Text licensed CC-BY 4.0; code licensed MIT.

unknown. Existing LLM evaluations emphasize accuracy on static question sets [11] or one-shot forecasting tournaments [10, 22]; they rarely enforce that the model sees only information that was available at prediction time, rarely sample the same process at many instants, and rarely compare against statistical baselines receiving exactly the same information. Live football forecasting is an unforgiving testbed for exactly these properties: targets are objectively defined, information arrives as a dense event stream, and the risk of post-hoc contamination—through data sources that silently encode final results—is pervasive.

This paper presents the design of a study that addresses this gap. We evaluate *Laya*, a typed, multilingual AI decision engine exposed through a software development kit (SDK), as a real-time probabilistic forecaster of football match outcomes: the final 1X2 result, the final score grouped into 17 buckets, total corners, and total yellow cards. The evaluation is *multi-instant*: each match is sampled at fixed cutoffs (pre-match, 15, 30, 45, 60, 75, 85 minutes) and immediately after salient events (goals, red cards, penalties, substitutions). It is *leakage-audited*: point-in-time integrity is enforced by construction and verified by automated tests executed on 100 % of snapshots. It is *fair*: every baseline receives the same information tier as *Laya* at the same cutoff. And it is *preregistered*: hypotheses, metrics, baselines, and blocking thresholds were frozen in this protocol before any test-set analysis, following open-science practice [19, 20].

Our contributions are:

1. a preregistered, fully reproducible evaluation protocol (v2.0.0) for an AI decision engine as a *multi-instant* probabilistic forecaster, with typed tasks, information tiers, and leakage tests on every snapshot;
2. an audit-first data architecture: immutable raw artifacts, deterministic SHA-256 identifiers, versioned state snapshots, and error-coded validity handling;
3. a nine-agent reference implementation whose agents communicate exclusively through hashed, versioned artifacts, enabling idempotent re-execution and third-party audit;
4. an open-source release (MIT code, CC-BY 4.0 text) that generalizes to other typed decision engines and other dynamic domains.

This document is the design paper (registered-report stage 1). It deliberately reports no empirical results; all results will be reported in stage 2 exactly as specified here, and analyses not specified here will be labeled exploratory.

2 Related Work

Statistical models of football outcomes. The reference models for association-football scores are decades old and remain strong. Maher [16] introduced independent Poisson models for home and away goals; Dixon and Coles [4] corrected the low-score dependence and introduced time-decayed maximum-likelihood estimation; Karlis and Ntzoufras [14] developed bivariate Poisson models for sports data; Rue and Salvesen [21] framed league prediction in a Bayesian dynamic model. For *in-play* forecasting, Dixon and Robinson [5] modeled the arrival of goals as a birth process with score-dependent intensities—an explicit model of how a distribution should evolve as the match unfolds, which is precisely the behavior we probe with event-driven snapshots. Corners and cards are less studied but are count variables for which Poisson and negative-binomial machinery applies directly.

Evaluating probabilistic forecasts. Brier’s 1950 score [1] and its decomposition [17] inaugurated distributional verification; the unified treatment of proper scoring rules by Gneiting and Raftery [7] and the calibration–sharpness principle of Gneiting et al. [8] define modern practice. For the ordered three-class 1X2 outcome, the ranked probability score is the natural strictly proper choice [7]. Calibration of predicted probabilities is usually summarized by the expected calibration error (ECE), whose binning choices are known to

matter [9, 18]. For count targets, Czado et al. [3] introduced the randomized probability integral transform (PIT) to correct the discreteness of PIT histograms, and discrete CRPS and log-score implementations are available in standard software [13]. Bookmaker odds are strong, hard-to-beat benchmarks when converted to implied probabilities with a margin-removal method fixed in advance [23]. Because snapshots of the same match share one realized outcome, our uncertainty quantification clusters at the match level, using bootstrap resampling of clusters [2, 6].

LLMs as probability assessors. Large-scale benchmarks measure the accuracy of LLM answers [11] but not the calibration of the full distributions they can be asked to output. A growing literature elicits verbalized or numerical confidence [15, 24] and reports promising aggregate accuracy in forecasting tournaments [10, 22]. What is missing is an evaluation of an LLM-based *engine*, queried repeatedly during a live process, under strict point-in-time information control, against matched statistical baselines, and with preregistered hypotheses. Our protocol imports the evaluation standards of the forecasting literature—proper scores, randomized PIT, coverage, clustered inference, and multiplicity control—into that setting.

Open science and preregistration. Large-scale replication failures [20] motivated preregistration and registered reports as a governance mechanism for confirmatory research [19]. We adopt the same discipline for an AI evaluation: hypotheses H1–H5, primary metrics, baseline definitions, split design, and blocking thresholds are frozen before the test set is inspected; this paper is the stage-1 (design) document of that registered report.

3 The Laya System Under Test

Laya¹ is a typed decision engine exposed through an SDK and a router that returns *structured probability distributions*, not free-form text. Two task types cover our targets: `choice`, which returns a categorical distribution over a versioned list of criteria, and `score`, which returns a distribution over an ordered support that may include a tail category (e.g., “21+”). The multilingual variant is used; canonical questions are written in English, and their exact wording, order, and category names are versioned—any change creates a new task version.

Three properties govern how the system enters the evaluation.

(i) Version pinning. Before any data collection, the following are recorded and frozen in the experiment manifest: exact package name and installed version, checkpoint hash or identifier, the engine variant used (the base or the multilingual one), maximum context length, Python version and dependency lockfile, router configuration, inference parameters (including any seed and batch settings), run identifier, and execution date. Architectural characteristics announced in vendor documentation are *not* treated as experimental facts until verified in the installed environment. The pinned version is used for all confirmatory analyses.

(ii) Contract validation before scale-up. A smoke test precedes collection and verifies: the real import name; the actual signature of the prediction entry point; the types accepted by the `questions` argument; the output format; the behavior on an invalid question; the reproducibility of two identical requests; and the conservation of probabilities and output keys. A deterministic repetition audit is then run on a stratified sample of at least 10 % of snapshots with $k = 5$ identical requests: if outputs are identical, a single query suffices in production; if they differ, either $k = 5$ is used for all snapshots of the main protocol, or a frozen

¹Laya is a third-party AI decision engine; it is evaluated as-is and is not modified by this study.

ID	Theme	Statement
H1	Temporal progression	The 1X2 log loss decreases as the cutoff moves from pre-match to 85 minutes.
H2	Calibration	Laya’s calibration (ECE and log loss) is better than that of the historical-frequency baseline at identical information.
H3	Event reaction	After a goal, the probability of the corresponding winner increases on average, all else equal.
H4	Robustness	Semantically equivalent paraphrases do not strongly modify the predictive distribution.
H5	Counts	For corners and cards, Laya is compared separately against (a) a historical-mean baseline and (b) an adapted count model.

Table 1: Preregistered confirmatory hypotheses (H1–H5).

aggregation rule is adopted *before* the test set—individual outputs are always retained. Inter-call variance, decision-flip frequency, and log-loss variation are reported.

(iii) Typed outputs and secondary confidence. Every response is stored raw (with hash) before parsing. Parsing succeeds only if the payload is valid JSON, contains exactly the documented keys, has finite probabilities in $[0, 1]$ summing to one within 10^{-6} , and has no missing or duplicate category; the expected value must be coherent with the declared tail convention. A failed validation is recorded with an error code and never silently repaired or renormalized. The *confidence* value optionally returned by Laya is a secondary output: it is stored and analyzed separately (calibration of confidence against the correctness of the most probable decision, and its relation to log loss), but it never substitutes for a probability in any metric. High confidence is not evidence of accuracy.

4 Evaluation Protocol

4.1 Research questions and preregistered hypotheses

The primary question is: *at a cutoff t , given only information available at t , does Laya produce predictive distributions that are better calibrated and/or more discriminative than the predefined baselines* for the final 1X2 result, the grouped final score, total corners, and total yellow cards? Secondary questions address improvement with match progression, reaction to salient events, stability under paraphrase and reordering, the information level required for a given quality, error concentration across matches and strata, and calibration-versus-discrimination trade-offs.

Five hypotheses are preregistered and must appear in the experiment manifest before the final test set is evaluated (Table 1). Any hypothesis not preregistered is exploratory and will be presented as such.

4.2 Data scope, sources, and conventions

The scope is five competitions—Premier League, La Liga, Serie A, Bundesliga, Ligue 1—over three complete seasons defined explicitly in `experiment_manifest.json`, with a power-oriented target of approximately 1,000 included matches. This figure is a power objective, not a quota: matches of insufficient quality are excluded, and the final counts of included, excluded, and missing matches are reported per competition and season.

One primary source is chosen before collection (candidates include StatsBomb, API-Football, FBref, or another documented source, subject to license verification); a secondary source verifies results and events

Task	Type	Support	Tail
1X2 final result	derived from score	home / draw / away	—
Grouped final score	choice	16 exact scores + other (17)	other
Total corners	score	0, . . . , 20	21+
Total yellow cards	score	0, . . . , 12	13+

Table 2: Primary prediction targets. Question wording, criteria lists, and their order are versioned; any modification creates a new task version.

but never silently completes the primary one. Every source record stores the provider, endpoint or file, source identifier, retrieval date, HTTP status or error code, SHA-256 hash of the raw content, and known license restrictions.

Inclusion requires: an officially finished match; no extra time; the final score present; events and statistics sufficient for the targets; timestamps precise enough for the cutoffs; and uniquely resolved home/away identities. *Exclusion with logging* applies to: stopped, postponed, replayed, or abandoned matches; extra time or shootouts; inconsistencies between final score and events; missing targets; impossible timestamps; unresolved duplicates; and post-match-only aggregates used where a live snapshot would be required. Excluded matches are never replaced after consulting results.

Counting conventions follow the protocol: goals—including penalties and own goals—count for the benefiting team in the official 90-minute-plus-stoppage score; shootout goals are not match goals; corners are counted per team and aggregated; yellow cards attributed to players on the pitch count toward the target, while cards shown to coaches, staff, substitutes, and the bench are excluded; a second yellow to the same player counts as such only where the source reliably supports it (with a preregistered sensitivity analysis); duplicate events are deduplicated on a documented key, never by untraceable manual deletion.

4.3 Targets and typed tasks

Table 2 summarizes the four primary tasks. The 1X2 distribution is derived from the joint score distribution,

$$P(1) = \sum_{h>a} P(h, a), \quad P(X) = \sum_{h=a} P(h, a), \quad P(2) = \sum_{h<a} P(h, a), \quad (1)$$

using the 90-minute-plus-stoppage score without extra time or shootouts. The grouped-score task uses 17 ordered buckets: the 16 exact scores with both components in $\{0, 1, 2, 3\}$ and an *other* bucket collecting any score with a component above 3. Because of the *other* class, this task is never described as an exact-score distribution. For higher resolution on goals, two separate marginal tasks (home goals and away goals, each 0–7 plus 8+) are added; these marginals are never recombined into a joint distribution without an explicitly validated method.

Corners use levels 0–20 plus a 21+ tail; yellow cards use 0–12 plus 13+. If the SDK cannot express an ordered tail class, the preregistered fallback is a censored target $\min(\text{total}, 20)$ (respectively $\min(\text{total}, 12)$), and expected values are then reported as censored expectations, never presented as uncensored. Tail thresholds may be raised *before* the experiment if descriptive analysis shows excessive tail mass; they may never be chosen after comparing results.

4.4 Snapshot plan and information tiers

Two families of snapshots sample each match. *Fixed* snapshots are taken at pre-match and at minutes 15, 30, 45, 60, 75, and 85 (`cutoffs_seconds` = [0, 900, 1800, 2700, 3600, 4500, 5100]), created only if the match is still in progress at the corresponding instant; the 45-minute snapshot represents the state at the end of the

Tier	Information content (available at the cutoff)
A	Pre-match: form, available standings, head-to-head, authorized odds, pre-match context.
B	A + minimal live: current score, cards, corners, dismissals, and the available event feed.
C	B + full live: shots, shots on target, possession, and xG where these were genuinely available at the cutoff.

Table 3: Information tiers. The tier label is part of the snapshot identity; ablation across tiers answers “how much live information is needed?”.

first period before interval events, unless a documented decision states otherwise. *Event-driven* snapshots are created immediately after every goal, red card, penalty (awarded or taken), and substitution; event type, timestamp, and identifier are stored in the snapshot metadata, and equal-timestamp events are handled by a documented source order plus one aggregated control version.

Each snapshot exists in three *information tiers*, fixed before analysis (Table 3). Tiers are different states of the same match and are never treated as independent observations in confidence intervals.

The state passed to Laya is a deterministic JSON serialization: sorted keys, fixed numeric precision, null for missing values, no uncontrolled free text, UTF-8 encoding. A missing live variable is never imputed from final statistics or from the realized outcome; accepted representations are null, a documented uniform omission (identical for Laya and the baselines), or a preregistered unavailable category. Missingness rates are reported per variable, source, competition, season, and cutoff.

4.5 Point-in-time integrity and leakage prevention

The **central rule** of the protocol is: at cutoff t , no artifact transmitted to Laya, no baseline feature, and no model-selection parameter may depend on information posterior to t for live data, or posterior to the pre-match cutoff for historical data. The final result is joined only after inference, at evaluation time.

Formally, each match carries T_k (kickoff time), T_{pre} (pre-match cutoff, equal to kickoff unless lineups or odds from an earlier time are explicitly included), and, for every snapshot, `cutoff_seconds` (elapsed playing time) and `cutoff_timestamp` (absolute time). Displayed minutes are insufficient when elapsed seconds are available; `elapsed_seconds` is preferred and stoppage-time handling is documented. Pre-match features are computed through a temporal view,

$$\text{available_timestamp} < T_{pre} \quad \text{and} \quad \text{match_id} \neq \text{current}, \quad (2)$$

never through whole-season or final-standing filters; the source match identifiers behind each feature are stored and verified. Live snapshots include only observations that were available no later than the snapshot cutoff: `available_timestamp` must not exceed `cutoff_timestamp`, and `elapsed_seconds` must not exceed `cutoff_seconds`. A statistic published only at the end of the match is never used as a live statistic even if it carries a “minute” column.

The snapshot state (`state_json`) must never contain: the final score or per-team final goals, corners, or cards; the realized result; any event posterior to the cutoff; any `full_time`, `final`, or `total_match` statistic; identifiers or texts revealing the target; result or evaluation file names; or outputs of a previous Laya prediction for the same match.

Three automated tests run on **100 % of snapshots**, inspecting keys, values, metadata, serialized text, and referenced paths: (i) no future events after the cutoff; (ii) no forbidden final fields; (iii) all pre-match source matches strictly earlier than the pre-match cutoff. A negative test artificially injects a future event and must cause the validator to fail. In addition, at least 5 % of snapshots are manually audited, stratified by competition, season, cutoff, and event type, checking coherence between raw source, canonical events, and the state transmitted to Laya; auditor decisions and justifications are logged.

4.6 Fair baselines

All baselines are trained or computed with data available at the same cutoff as Laya’s snapshot, and each receives an `information_tier` feature table equivalent to Laya’s. A baseline that would use variables unavailable to Laya is excluded from the main comparison and reported as auxiliary analysis.

Minimal baselines: the uniform distribution; the historical frequency in the training window (per competition, optionally per season); persistence of the current score as the final score (a deliberately naive baseline, converted to a distribution by a preregistered rule); and persistence of counts (current count plus the historical average of the remainder, with the remainder precisely defined). A point-estimate baseline is never compared to a distribution without a documented conversion method.

Statistical models: independent or bivariate Poisson for goals; a Dixon–Coles-type model where justified; Poisson or negative binomial for corners and cards; multinomial logistic regression for 1X2; and, optionally, a tabular gradient-boosting model used only with a temporal split. Hyperparameters are fixed on the validation block only—never on the final test set.

Bookmaker odds: pre-match odds are a strong baseline at the pre-match snapshot only, unless time-stamped live odds are available and integrated identically for all systems. Odds are converted to implied probabilities and the margin is removed by a method fixed before analysis; an odds value published after the cutoff is never used.

4.7 Temporal split and dependence

No random split is performed at the snapshot level. Training uses the oldest periods, validation the immediately following period, and the final test the most recent period—never consulted for model choices. With only three seasons available, a documented rolling-origin scheme with a final temporal holdout is used. Matches played on the same day remain in the same block whenever feature availability could create dependence. Because all snapshots of a match share one realized outcome—and event-driven snapshots are even more dependent—per-snapshot metrics are descriptive only; standard errors and intervals are clustered by `match_id`.

A power analysis precedes final collection: primary metric per target, minimal difference of interest, expected exclusion rate, required number of matches, number of competitions and seasons, and the maximum number of confirmatory tests. If power is insufficient, an imprecise estimate is reported rather than a claim of equivalence from a non-significant result.

4.8 Metrics

1X2 (ordered three-class). Multiclass log loss with clipping floor $\varepsilon = 10^{-15}$,

$$\text{LL} = -\frac{1}{N} \sum_{i=1}^N \log(\max\{\hat{p}_i(y_i), \varepsilon\}); \quad (3)$$

multiclass Brier score; accuracy of the most probable class; the ranked probability score, appropriate for the ordered home–draw–away structure; ECE with fixed bin count and method; reliability curves; and breakdowns by cutoff, competition, season, and tier. AUC is not a primary metric here; if reported, it is auxiliary with an explicit one-vs-rest method.

Grouped score (17 buckets). Log loss over the 17 categories (never called “exact-score log loss”), multiclass Brier, RPS under a defined coherent order, top-bucket accuracy, and calibration of frequent buckets and of other.

Counts (corners, cards). MAE and RMSE of the expected value only when compatible with the tail convention; discrete log score; discrete CRPS; Poisson or negative-binomial deviance where applicable; coverage and mean width of 50 %, 80 %, and 95 % predictive intervals; randomized PIT for discrete variables [3]; and tail mass (21+, 13+). If a tail is censored, the censored-target metrics are reported separately with the interpretation limitation stated.

Calibration. For every distribution: ECE with bin count, maximum calibration error, reliability curves, randomized PIT histograms for counts, empirical coverage, and the Brier decomposition where implemented. The binning method is frozen; a quantile-binning sensitivity analysis may be added.

4.9 Statistical inference

Model comparison uses a stratified bootstrap *clustered by match*: matches are resampled with replacement (at least 2,000 replicates where the budget allows), and each metric is recomputed over all snapshots of the resampled matches. For a pair of models, the per-match difference in metric contributions is computed once, and the bootstrap is applied to those differences, yielding a mean difference, a 95 % interval, and an effect size. Multiplicity is controlled by Holm–Bonferroni [12] within preregistered families of tests; the family structure is frozen in the manifest. A non-significant test is never presented as evidence of equality.

4.10 Robustness, event reactions, and near-identical states

Paraphrase robustness uses a small, versioned set of semantically equivalent formulations with identical state content and categories. Measured quantities: mean absolute probability change, Jensen–Shannon and total-variation distances, expected-value change, most-probable-class flip rate, and log-loss variation. A single 1X2 decision does not measure the robustness of a distribution. *Format robustness* separately tests JSON field order, field-name variants, compact versus indented serialization, French versus English (only for the multilingual variant), and the presence of a fixed descriptive dictionary—each variant preserving semantic content and context budget.

Event reactions compare before/after snapshots within a predefined window for each admissible event. Expected directions are preregistered without automatic causal interpretation: after a home goal, $P(1)$ should rise on average; after an away goal, $P(2)$; after a home red card, $P(1)$ should fall and $P(2)$ may rise; a penalty effect should favor the benefiting team whenever the penalty outcome is not posterior to the cutoff. Reactions are also measured on the corners and cards distributions, without assuming that an event must improve all objectives. *Near-identical snapshot pairs* (states differing little but with different times or events) are screened for unjustified oscillations, class changes without new information, excessive concentration, and incoherence between the current score and the final distribution. These analyses are descriptive and do not replace out-of-sample evaluation.

4.11 Quality gates and blocking criteria

The final test is blocked—the confirmatory analysis may not proceed—if any of the following holds: more than 5 % of snapshots contain leakage; more than 10 % of matches are missing in a mandatory stratum; the rate of invalid Laya responses exceeds 5 % without a sensitivity analysis; a checkpoint or task is not fully versioned; or the provenance of a target or feature cannot be audited. These thresholds may be modified only in the preregistered manifest. Every aggregation publishes its denominator and invalidity rate; idempotent re-runs never silently alter validated artifacts.

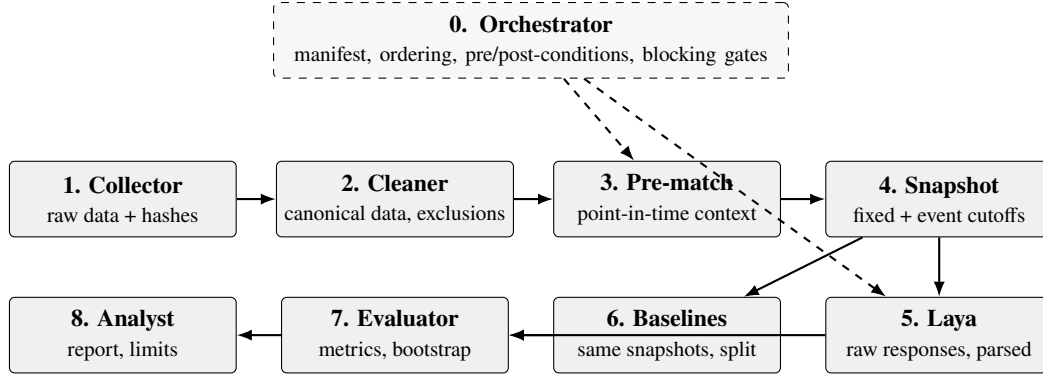


Figure 1: Nine-agent pipeline. Agents communicate exclusively through hashed, versioned artifacts; the orchestrator validates pre- and post-conditions of every step and stops the pipeline on blocking errors.

Agent	Role (outputs)	Blocking condition
Orchestrator	Manifest, ordering, run journal	Any blocking error downstream
Collector	Raw responses, call logs, hashes, coverage report	Bounded retries only; no transformation
Cleaner	Canonical data, cleaning report, exclusions	Identifier, timing, or score incoherence
Pre-match	Point-in-time context, source-match lists	Any feature with a future date
Snapshot	Fixed + event snapshots, state hashes	Any failed leakage test
Laya	Raw responses, normalized predictions, latency, repetition audit	Invalid response isolated, never repaired
Baselines	Baseline distributions, fitted parameters	Any training on the final test
Evaluator	Per-prediction contributions, aggregates, bootstrap, calibration	No aggregation without denominator report
Analyst	Final report, limits, reproducible annexes	Confirmatory/exploratory separation required

Table 4: The nine agents of the reference architecture.

5 Multi-Agent Reference Architecture

The protocol is executed by nine specialized agents that never communicate directly: they read and write *versioned artifacts*—JSON carrying a schema version, experiment and run identifiers, producer name, timestamp, input hashes, output hash, status (validated, warning, or blocked), record count, and warning/error lists. Agents are idempotent: re-running a step for an existing `run_id` neither duplicates rows nor modifies validated artifacts. Identifiers are deterministic, e.g.,

$$\text{match_id} = \text{SHA256}(\text{competition} : \text{season} : \text{kickoff} : \text{home} : \text{away}), \quad (4)$$

and snapshots are keyed by match, cutoff, tier, and variant. Figure 1 shows the pipeline; Table 4 summarizes responsibilities and blocking conditions.

Every Laya response is classified by exactly one error code:

OK • NETWORK_RETRY_EXHAUSTED • INVALID_JSON • INVALID_SCHEMA
 INVALID_PROBABILITY • INVALID_PROBABILITY_SUM • MISSING_CATEGORY
 LEAKAGE_DETECTED • CONTEXT_TOO_LONG • SDK_ERROR

Network errors may be retried with bounded exponential backoff; every attempt is logged and no retry ever replaces the original response. Parsing, schema, and leakage failures are validity errors and are never auto-repaired without diagnosis. Ten per-agent log files plus a shared error log record timestamp, agent, run identifier, entity identifier, action, status, and a short message—with no personal data; player identifiers are pseudonymized wherever retained. The fixed execution order is collector, cleaner, pre-match, snapshot, Laya, baselines, evaluator, analyst; each step verifies its prerequisite artifacts and writes an output manifest with status *validated*, *warning*, or *blocked*.

The protocol is considered *executed as specified* when: a complete run can be launched from the manifest; all artifacts are hashed and attributed; 100% of snapshots pass the leakage tests; all responses are classified valid or invalid with a code; every metric publishes its denominator; the baselines use an equivalent temporal split; intervals are clustered by match; the *other* and tail limitations are visible in the report; the final test results did not modify the protocol; and a third party can reproduce at least the primary metrics from the stored artifacts without new API calls.

6 Experimental Status

This article is the stage-1 document of a registered report. The protocol (v2.0.0), the preregistration package, and the reference implementation are published and frozen before the final test set is collected or inspected: preregistration at [OSF DOI]; code at [GITHUB URL]. The pinned Laya version and checkpoint will be recorded in the frozen experiment manifest at execution time. Data collection, prediction, and evaluation follow after preregistration; results will be reported in stage 2 exactly as specified here. The reproducibility package retains: source code and commit hashes; dependency lockfile and Python version; the complete experiment manifest; seeds and model configurations; the exact Laya version and checkpoint hash; SHA-256 hashes of raw and canonical data; exact questions and criteria; raw Laya responses; snapshot-generation and metric scripts; exclusion and error tables; the sample-size report; and the dates and time zones of all timestamps. A reproduction must be able to restart from the raw files without calling the external API again, except for steps explicitly marked non-reproducible.

7 Discussion and Limitations

The design has known limits, stated here rather than after the results. *Dependence*: snapshots of a match share one realized outcome; clustered bootstrap mitigates but does not eliminate the resulting inferential fragility, and event-driven snapshots are more dependent still. *Coarse categories*: the *other* bucket and the censored 21+ and 13+ tails bound the interpretability of score distributions and expected values. *Live-data reality*: live statistics are published with delays and revisions; we model availability as it actually occurred, so tier-C snapshots may carry less information than an idealized real-time feed. *Scope*: five competitions over three seasons; nothing here supports a universal claim about Laya, about other leagues, or about LLM-based engines in general. *Causality*: event-reaction contrasts are descriptive associations in preregistered directions, not causal effects. *Use*: the study evaluates predictive quality only; it is not betting advice and reports no profitability. *Validity handling*: invalid responses are counted, never repaired; a high invalidity rate can block the analysis rather than be hidden. *Sources*: public sports data can be corrected after publication and carry license restrictions; apparent quality may reflect coverage biases of the chosen sources. Finally, Laya’s announced characteristics are treated as unverified until confirmed in the installed environment.

8 Conclusion

We presented a preregistered, multi-instant, leakage-audited protocol that evaluates a typed AI decision engine—Laya—as a real-time probabilistic forecaster of football outcomes, against statistical baselines receiving identical point-in-time information, with clustered inference, multiplicity control, and preregistered hypotheses. The design converts the standards of probabilistic-forecast evaluation into an executable, auditable, nine-agent pipeline whose every artifact is hashed and every aggregation publishes its denominator. Whatever the stage-2 outcome, the protocol and its open-source implementation provide a reusable template for evaluating probabilistic AI systems in dynamic, information-asymmetric environments.

Acknowledgements

This study protocol, the reference implementation, and this manuscript were developed with the assistance of an AI system (Super Z, Z.ai) under the direction of the author, who verified all content and takes full responsibility for it. This disclosure follows emerging academic transparency norms regarding the use of AI assistance in research.

Data and Code Availability

The reference implementation (Python, SQLite) is open source under the MIT license at [GITHUB URL]; the protocol text and this manuscript are licensed under CC-BY 4.0. The preregistration is available at [OSF DOI]. Raw match data are not redistributed in the repository; their provenance, licenses, and SHA-256 hashes are documented so that the full pipeline can be re-executed from authorized sources. Laya is a third-party system, evaluated as-is.

References

- [1] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [2] A. Colin Cameron and Douglas L. Miller. A practitioner’s guide to cluster-robust inference. *Journal of Human Resources*, 50(2):317–372, 2015.
- [3] Claudia Czado, Tilmann Gneiting, and Leonhard Held. Predictive model assessment for count data. *Biometrics*, 65(4):1254–1261, 2009.
- [4] Mark J. Dixon and Stuart G. Coles. Modelling association football scores and inferences from match scores. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 46(2):265–280, 1997.
- [5] Mark J. Dixon and Michael E. Robinson. A birth process model for association football matches. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 47(3):523–538, 1998.
- [6] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [7] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

- [8] Tilmann Gneiting, Fadoua Balabdaoui, and Adrian E. Raftery. Probabilistic forecasts, calibration and sharpness. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(2):243–268, 2007.
- [9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- [10] Danny Halawi, Fred Zhang, Chen Yueh-Han, and Jacob Steinhardt. Approaching human-level forecasting with language models. *arXiv preprint arXiv:2402.18563*, 2024.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [12] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [13] Alexander Jordan, Fabian Krüger, and Sebastian Lerch. Evaluating probabilistic forecasts with scoringRules. *Journal of Statistical Software*, 90(12):1–37, 2019.
- [14] Dimitris Karlis and Ioannis Ntzoufras. Analysis of sports data by using bivariate Poisson models. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 52(3):381–393, 2003.
- [15] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022. arXiv:2205.14334.
- [16] Michael J. Maher. Modelling association football scores. *Statistica Neerlandica*, 36(3):109–118, 1982.
- [17] Allan H. Murphy. A new vector partition of the probability score. *Journal of Applied Meteorology*, 12(4):595–600, 1973.
- [18] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milo Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, pages 2901–2907, 2015.
- [19] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- [20] Open Science Collaboration. Estimating the reproducibility of psychological science. *Science*, 349(6251):aac4716, 2015.
- [21] Håvard Rue and Ørnulf Salvesen. Prediction and retrospective analysis of soccer matches in a league. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 49(3):399–418, 2000.
- [22] Philipp Schoenegger, Sijeong Park, PH Kuznetsova, and James T. O’Brien. Wisdom of the Silicon crowd: LLM ensemble prediction capabilities rival human crowd accuracy. *Science Advances*, 10(28), 2024.
- [23] Martin Spann and Bernd Skiera. Sports forecasting: a comparison of the forecast accuracy of prediction markets, betting odds and tipsters. *Journal of Forecasting*, 28(5):55–65, 2009.
- [24] Miao Xiong, Zhiyuan Hu, Xiaofei Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. *International Conference on Learning Representations (ICLR)*, 2024.